HDOC Day-57

Maximum Subarray Sum Using Kadane's Algorithm, Functions, and Pointers in C

Aim

To write a C program using a variable-length integer array, user-defined functions, pointers, and Kadane's algorithm to find the maximum sum of any non-empty contiguous subarray. The program must return the largest (least negative) element when all elements are negative.

Algorithm

1. Start the program and read the size of the array.

2. Validate that size is positive; otherwise display an error and stop.

3. Declare a variable-length integer array arr[size] and read its elements.

4. Pass arr and size to max_subarray_sum() using the array address.

5. Initialize currentSum and maxSum with the first element of the array.

6. For each remaining element, set currentSum to the greater of the element itself and currentSum plus that element.

7. Update maxSum whenever currentSum is greater than maxSum.

8. Display maxSum. Initializing with the first element ensures the subarray is non-empty, even for all-negative arrays.

9. Stop the program.

C Program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int size);

int main()
{
    int size, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];
    printf("Enter %d array elements: ", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    max_subarray_sum(arr, size);
    return 0;
}

/* Function to find maximum subarray sum */
void max_subarray_sum(int *arr, int size)
{
    int i;
    long long currentSum, maxSum;

    currentSum = *(arr);
    maxSum = *(arr);

    for (i = 1; i < size; i++)
    {
        if (currentSum + *(arr + i) > *(arr + i))
        {
            currentSum = currentSum + *(arr + i);
        }
        else
        {
            currentSum = *(arr + i);
        }

        if (currentSum > maxSum)
        {
            maxSum = currentSum;
        }
    }

    printf("\nMaximum Subarray Sum: %lld\n", maxSum);
}
```

Explanation

The function max_subarray_sum(int *arr, int size) receives the address of the first array element. The expression *(arr + i) accesses arr[i]. Kadane's algorithm either extends the previous contiguous subarray or starts a new one at the current element. Its time complexity is O(n), and its auxiliary space complexity is O(1).

Example array: -2 1 -3 4 -1 2 1 -5 4. The maximum-sum contiguous subarray is 4 -1 2 1, with sum 6.

Test Case Table

Case	Input Array	Expected Result
1	-2 1 -3 4 -1 2 1 -5 4	6
2	1 2 3 4 5	15
3	-5 -2 -8 -1 -4	-1
4	5 -2 3 4	10
5	-1 -2 -3 -4	-1
6	10	10
7	-7	-7
8	0 0 0 0	0
9	2 -1 2 3 -9 5	6
10	size = 0	Invalid array size

Sample Outputs

Mixed Positive and Negative Elements

```
Enter the size of array: 9
Enter 9 array elements: -2 1 -3 4 -1 2 1 -5 4

Maximum Subarray Sum: 6
```

All Positive Elements

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5

Maximum Subarray Sum: 15
```

All Negative Elements

```
Enter the size of array: 5
Enter 5 array elements: -5 -2 -8 -1 -4

Maximum Subarray Sum: -1
```

Single Negative Element

```
Enter the size of array: 1
Enter 1 array elements: -7

Maximum Subarray Sum: -7
```

Invalid Array Size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program was designed using variable-length arrays, pointers, user-defined functions, loops, conditional statements, and Kadane's algorithm. It finds the maximum sum of a non-empty contiguous subarray and correctly handles mixed, all-positive, all-negative, and single-element inputs.